Processador MIPS com Pipeline em Verilog

Ana Clara Eid Freire (22401684), Jaide Fernando de Carvalho Zardin (22350981), Manuela Lopes Figueira (22400983)

Curso de Engenharia da Computação — Universidade Federal do Amazonas (UFAM)

Arquitetura de Sistemas Digitais e Laboratório — Relatório da 1ª entrega parcial — 11/09/2026

Resumo — Este relatório apresenta o status da primeira entrega do projeto de implementação, em Verilog, de um processador MIPS com pipeline de cinco estágios, com tratamento de perigos (hazards) de dados, controle e estruturais. São definidos o contexto e os objetivos do projeto, a plataforma de desenvolvimento, a memória de programa utilizada para validação — já traduzida para assembly MIPS, codificada em binário por um montador desenvolvido pela própria equipe e verificada por simulação de execução completa —, o plano de trabalho da equipe e a gestão do projeto por meio do Trello.

Palavras-chave — *Processador MIPS; Pipeline; Verilog; FPGA; Arquitetura de Computadores.*

I. INTRODUÇÃO, CONTEXTO E OBJETIVOS

Um processador monociclo executa cada instrução em um único ciclo de clock, cuja duração é limitada pelo caminho crítico da instrução mais lenta (tipicamente lw). Isso subutiliza o hardware durante instruções mais simples e impede aumentar a frequência de operação.

O pipeline resolve esse problema dividindo a execução em estágios (IF, ID, EX, MEM, WB) que processam instruções diferentes simultaneamente: o clock passa a ser limitado pelo estágio mais lento — não pela instrução mais lenta — e, em regime estável, uma instrução é concluída por ciclo.

O ganho não é proporcional ao número de estágios porque (i) perigos de dados, controle e estruturais forçam bolhas (stalls) que interrompem esse fluxo ideal; (ii) os registradores de pipeline entre estágios adicionam um overhead fixo de tempo a cada ciclo; e (iii) o balanceamento entre estágios raramente é perfeito — o estágio mais lento dita o clock de todos os outros.

Este projeto se propõe a implementar, em Verilog, um processador MIPS com pipeline de 5 estágios, tratando os três tipos de perigo, com todos os blocos funcionais parametrizados e modularizados. O processador será embarcado em uma FPGA e validado com o procedimento de ordenação por inserção (sort/swap) especificado pelo professor da disciplina [1].

Critério objetivo de sucesso: o vetor de teste é ordenado corretamente pelo hardware — o resultado observado na FPGA (ou em simulação, antes da síntese) confere com a saída esperada do sort() em C — com cada bloco funcional validado isoladamente por testbench antes da integração.

II. PLATAFORMA E MATERIAIS

FPGA: Altera/Intel DE2-115 (Cyclone IV, EP4CE115).

Software de síntese: Intel Quartus Prime (versão a confirmar junto ao laboratório da disciplina).

Simulador: ModelSim-Altera Starter Edition — distribuído junto ao Quartus para a família Cyclone IV, dispensando licença separada.

Linguagem de descrição de hardware: Verilog.

Controle de versão: Git / GitHub (repositório privado da equipe).

Gestão de projeto: Trello.

III. MEMÓRIA DE PROGRAMA

A convenção de registradores para o programa de teste, fixada pelo enunciado, é: v (vetor) em $a0, n (tamanho) em $a1, i em $s0 e j em $s1. O código-fonte em C fornecido é:

```
void sort (int v[], int n) {
  int i, j;
  for (i = 0; i < n; i += 1) {
    for (j = i-1; j >= 0 && v[j] > v[j+1]; j -= 1)
{
      swap(v, j);
    }
  }
}

void swap (int v[], int k) {
  int temp;
  temp   = v[k];
  v[k]   = v[k+1];
  v[k+1] = temp;
}
```

As três funções foram traduzidas manualmente para assembly MIPS. O prólogo de sort() salva $ra, $s0, $s1, $s2 e $s3 na pilha — $ra porque a chamada interna a swap() o sobrescreveria, e $s2/$s3 guardam cópias de n e v, que de outra forma seriam destruídas pela passagem de argumentos ($a0/$a1) para swap(). A condição composta do laço interno (j ≥ 0 && v[j] > v[j+1]) foi decomposta em duas checagens sequenciais, cada uma podendo encerrar o laço.

Um ponto de atenção identificado durante a verificação: como o contador de programa (PC) do processador inicia no endereço 0 e executa a primeira instrução física do arquivo montado — não um rótulo —, foi necessário inserir uma instrução de salto incondicional (j main) como primeira instrução do programa; sem ela, a execução iniciaria dentro do corpo de swap(), antes de main() configurar os dados de teste.

O montador (tools/assembler.py), desenvolvido pela equipe, converte o assembly nos três formatos de instrução do MIPS — R (op·rs·rt·rd·shamt·funct), I (op·rs·rt·imediato) e J (op·endereço) — usando duas passagens: a primeira mapeia rótulos para endereços; a segunda codifica cada instrução, resolvendo desvios (beq/bne, relativos ao PC seguinte, contados em instruções) e saltos (j/jal, endereço absoluto dividido por 4).

Verificação: o programa foi validado em duas etapas independentes. Primeiro, a lógica do algoritmo foi reproduzida e executada isoladamente, confirmando a ordenação correta de um vetor de teste. Em seguida — verificação mais rigorosa — o próprio arquivo binário gerado (.mif) foi decodificado bit a bit e executado por um interpretador que reproduz o ciclo de busca-decodificação-execução do processador a partir do endereço 0, exatamente como o hardware fará. Ambas as simulações resultaram no vetor de teste {5,3,8,1,9,2,7,4} corretamente ordenado como {1,2,3,4,5,7,8,9}. O programa completo tem 63 instruções; a codificação integral está na Tabela I.

TABELA I

MEMÓRIA DE PROGRAMA CODIFICADA EM BINÁRIO (mem/programa.asm → mem/programa.mif, 63 instruções)

Endereço	Rótulo	Instrução (assembly)	Fmt.	Campos	Codificação binária (32 bits)
0x00		j main	J	op=000010 endereco=00000000000000000000101010	0000 1000 0000 0000 0000 0000 0010 1010
0x04	swap	sll $t1, $a1, 2	R	op=000000 rs=00000 rt=00101 rd=01001 sh=00010 fn=000000	0000 0000 0000 0101 0100 1000 1000 0000
0x08		add $t1, $a0, $t1	R	op=000000 rs=00100 rt=01001 rd=01001 sh=00000 fn=100000	0000 0000 1000 1001 0100 1000 0010 0000
0x0C		lw $t2, 0($t1)	I	op=100011 rs=01001 rt=01010 imm=0000000000000000	1000 1101 0010 1010 0000 0000 0000 0000
0x10		lw $t0, 4($t1)	I	op=100011 rs=01001 rt=01000 imm=0000000000000100	1000 1101 0010 1000 0000 0000 0000 0100
0x14		sw $t0, 0($t1)	I	op=101011 rs=01001 rt=01000 imm=0000000000000000	1010 1101 0010 1000 0000 0000 0000 0000
0x18		sw $t2, 4($t1)	I	op=101011 rs=01001 rt=01010 imm=0000000000000100	1010 1101 0010 1010 0000 0000 0000 0100
0x1C		jr $ra	R	op=000000 rs=11111 rt=00000 rd=00000 sh=00000 fn=001000	0000 0011 1110 0000 0000 0000 0000 1000
0x20	sort	addi $sp, $sp, -20	I	op=001000 rs=11101 rt=11101 imm=1111111111101100	0010 0011 1011 1101 1111 1111 1110 1100
0x24		sw $ra, 16($sp)	I	op=101011 rs=11101 rt=11111 imm=0000000000010000	1010 1111 1011 1111 0000 0000 0001 0000
0x28		sw $s3, 12($sp)	I	op=101011 rs=11101 rt=10011 imm=0000000000001100	1010 1111 1011 0011 0000 0000 0000 1100
0x2C		sw $s2, 8($sp)	I	op=101011 rs=11101 rt=10010 imm=0000000000001000	1010 1111 1011 0010 0000 0000 0000 1000
0x30		sw $s1, 4($sp)	I	op=101011 rs=11101 rt=10001 imm=0000000000000100	1010 1111 1011 0001 0000 0000 0000 0100
0x34		sw $s0, 0($sp)	I	op=101011 rs=11101 rt=10000 imm=0000000000000000	1010 1111 1011 0000 0000 0000 0000 0000
0x38		add $s3, $a0, $zero	R	op=000000 rs=00100 rt=00000 rd=10011 sh=00000 fn=100000	0000 0000 1000 0000 1001 1000 0010 0000
0x3C		add $s2, $a1, $zero	R	op=000000 rs=00101 rt=00000 rd=10010 sh=00000 fn=100000	0000 0000 1010 0000 1001 0000 0010 0000
0x40		addi $s0, $zero, 0	I	op=001000 rs=00000 rt=10000 imm=0000000000000000	0010 0000 0001 0000 0000 0000 0000 0000
0x44	for1tst	slt $t0, $s0, $s2	R	op=000000 rs=10000 rt=10010 rd=01000 sh=00000 fn=101010	0000 0010 0001 0010 0100 0000 0010 1010
0x48		beq $t0, $zero, exit1	I	op=000100 rs=01000 rt=00000 imm=0000000000010000	0001 0001 0000 0000 0000 0000 0001 0000
0x4C		addi $s1, $s0, -1	I	op=001000 rs=10000 rt=10001 imm=1111111111111111	0010 0010 0001 0001 1111 1111 1111 1111
0x50	for2tst	slti $t0, $s1, 0	I	op=001010 rs=10001 rt=01000 imm=0000000000000000	0010 1010 0010 1000 0000 0000 0000 0000
0x54		bne $t0, $zero, exit2	I	op=000101 rs=01000 rt=00000 imm=0000000000001011	0001 0101 0000 0000 0000 0000 0000 1011
0x58		sll $t1, $s1, 2	R	op=000000 rs=00000 rt=10001 rd=01001 sh=00010 fn=000000	0000 0000 0001 0001 0100 1000 1000 0000

Endereço	Rótulo	Instrução (assembly)	Fmt.	Campos	Codificação binária (32 bits)
0x5C		add $t2, $s3, $t1	R	op=000000 rs=10011 rt=01001 rd=01010 sh=00000 fn=100000	0000 0010 0110 1001 0101 0000 0010 0000
0x60		lw $t3, 0($t2)	I	op=100011 rs=01010 rt=01011 imm=0000000000000000	1000 1101 0100 1011 0000 0000 0000 0000
0x64		lw $t4, 4($t2)	I	op=100011 rs=01010 rt=01100 imm=0000000000000100	1000 1101 0100 1100 0000 0000 0000 0100
0x68		slt $t0, $t4, $t3	R	op=000000 rs=01100 rt=01011 rd=01000 sh=00000 fn=101010	0000 0001 1000 1011 0100 0000 0010 1010
0x6C		beq $t0, $zero, exit2	I	op=000100 rs=01000 rt=00000 imm=0000000000000101	0001 0001 0000 0000 0000 0000 0000 0101
0x70		add $a0, $s3, $zero	R	op=000000 rs=10011 rt=00000 rd=00100 sh=00000 fn=100000	0000 0010 0110 0000 0010 0000 0010 0000
0x74		add $a1, $s1, $zero	R	op=000000 rs=10001 rt=00000 rd=00101 sh=00000 fn=100000	0000 0010 0010 0000 0010 1000 0010 0000
0x78		jal swap	J	op=000011 endereco=00000000000000000000000001	0000 1100 0000 0000 0000 0000 0000 0001
0x7C		addi $s1, $s1, -1	I	op=001000 rs=10001 rt=10001 imm=1111111111111111	0010 0010 0011 0001 1111 1111 1111 1111
0x80		j for2tst	J	op=000010 endereco=00000000000000000000010100	0000 1000 0000 0000 0000 0000 0001 0100
0x84	exit2	addi $s0, $s0, 1	I	op=001000 rs=10000 rt=10000 imm=0000000000000001	0010 0010 0001 0000 0000 0000 0000 0001
0x88		j for1tst	J	op=000010 endereco=00000000000000000000010001	0000 1000 0000 0000 0000 0000 0001 0001
0x8C	exit1	lw $s0, 0($sp)	I	op=100011 rs=11101 rt=10000 imm=0000000000000000	1000 1111 1011 0000 0000 0000 0000 0000
0x90		lw $s1, 4($sp)	I	op=100011 rs=11101 rt=10001 imm=0000000000000100	1000 1111 1011 0001 0000 0000 0000 0100
0x94		lw $s2, 8($sp)	I	op=100011 rs=11101 rt=10010 imm=0000000000001000	1000 1111 1011 0010 0000 0000 0000 1000
0x98		lw $s3, 12($sp)	I	op=100011 rs=11101 rt=10011 imm=0000000000001100	1000 1111 1011 0011 0000 0000 0000 1100
0x9C		lw $ra, 16($sp)	I	op=100011 rs=11101 rt=11111 imm=0000000000010000	1000 1111 1011 1111 0000 0000 0001 0000
0xA0		addi $sp, $sp, 20	I	op=001000 rs=11101 rt=11101 imm=0000000000010100	0010 0011 1011 1101 0000 0000 0001 0100
0xA4		jr $ra	R	op=000000 rs=11111 rt=00000 rd=00000 sh=00000 fn=001000	0000 0011 1110 0000 0000 0000 0000 1000
0xA8	main	addi $sp, $zero, 1024	I	op=001000 rs=00000 rt=11101 imm=0000010000000000	0010 0000 0001 1101 0000 0100 0000 0000
0xAC		addi $a0, $zero, 0	I	op=001000 rs=00000 rt=00100 imm=0000000000000000	0010 0000 0000 0100 0000 0000 0000 0000
0xB0		addi $t0, $zero, 5	I	op=001000 rs=00000 rt=01000 imm=0000000000000101	0010 0000 0000 1000 0000 0000 0000 0101
0xB4		sw $t0, 0($a0)	I	op=101011 rs=00100 rt=01000 imm=0000000000000000	1010 1100 1000 1000 0000 0000 0000 0000
0xB8		addi $t0, $zero, 3	I	op=001000 rs=00000 rt=01000 imm=0000000000000011	0010 0000 0000 1000 0000 0000 0000 0011
0xBC		sw $t0, 4($a0)	I	op=101011 rs=00100 rt=01000	1010 1100 1000 1000 0000 0000 0000 0100

Endereço	Rótulo	Instrução (assembly)	Fmt.	Campos	Codificação binária (32 bits)
				imm=0000000000000100	
0xC0		addi $t0, $zero, 8	I	op=001000 rs=00000 rt=01000 imm=0000000000001000	0010 0000 0000 1000 0000 0000 0000 1000
0xC4		sw $t0, 8($a0)	I	op=101011 rs=00100 rt=01000 imm=0000000000001000	1010 1100 1000 1000 0000 0000 0000 1000
0xC8		addi $t0, $zero, 1	I	op=001000 rs=00000 rt=01000 imm=0000000000000001	0010 0000 0000 1000 0000 0000 0000 0001
0xCC		sw $t0, 12($a0)	I	op=101011 rs=00100 rt=01000 imm=0000000000001100	1010 1100 1000 1000 0000 0000 0000 1100
0xD0		addi $t0, $zero, 9	I	op=001000 rs=00000 rt=01000 imm=0000000000001001	0010 0000 0000 1000 0000 0000 0000 1001
0xD4		sw $t0, 16($a0)	I	op=101011 rs=00100 rt=01000 imm=0000000000010000	1010 1100 1000 1000 0000 0000 0001 0000
0xD8		addi $t0, $zero, 2	I	op=001000 rs=00000 rt=01000 imm=0000000000000010	0010 0000 0000 1000 0000 0000 0000 0010
0xDC		sw $t0, 20($a0)	I	op=101011 rs=00100 rt=01000 imm=0000000000010100	1010 1100 1000 1000 0000 0000 0001 0100
0xE0		addi $t0, $zero, 7	I	op=001000 rs=00000 rt=01000 imm=0000000000000111	0010 0000 0000 1000 0000 0000 0000 0111
0xE4		sw $t0, 24($a0)	I	op=101011 rs=00100 rt=01000 imm=0000000000011000	1010 1100 1000 1000 0000 0000 0001 1000
0xE8		addi $t0, $zero, 4	I	op=001000 rs=00000 rt=01000 imm=0000000000000100	0010 0000 0000 1000 0000 0000 0000 0100
0xEC		sw $t0, 28($a0)	I	op=101011 rs=00100 rt=01000 imm=0000000000011100	1010 1100 1000 1000 0000 0000 0001 1100
0xF0		addi $a1, $zero, 8	I	op=001000 rs=00000 rt=00101 imm=0000000000001000	0010 0000 0000 0101 0000 0000 0000 1000
0xF4		jal sort	J	op=000011 endereco=00000000000000000000001000	0000 1100 0000 0000 0000 0000 0000 1000
0xF8	end	j end	J	op=000010 endereco=00000000000000000000111110	0000 1000 0000 0000 0000 0000 0011 1110

IV. PLANO DE TRABALHO

Divisão por bloco funcional

A divisão do trabalho é feita por bloco funcional, e não por fase do pipeline: dividir por fase (ex.: uma pessoa para o monociclo, outra para o pipeline) criaria dependência sequencial entre os integrantes. Dividindo por bloco, os três trabalham em paralelo desde o início.

Responsável	Blocos
Pessoa A — Datapath e memória	Banco de registradores; memória de instruções; memória de dados; extensor de sinal; multiplexadores do datapath
Pessoa B (Ana C. E. Freire) — ULA e controle	ALU parametrizada; unidade de controle principal; ALU control; somador de PC / lógica de branch-jump
Pessoa C — Pipeline, hazards e integração	Registradores de pipeline (IF/ID, ID/EX, EX/MEM, MEM/WB); unidade de forwarding; detecção de hazard (stall); integração final e testbench completo

A definição de quais integrantes ocupam os papéis de Pessoa A e Pessoa C está em aberto e será fechada até o início da 2ª etapa (09/10/2026). Todos os integrantes participam da fase de testes e da escrita do relatório.

Cronograma

Período	Meta
Até 11/09	Ambiente configurado; FPGA confirmada (DE2-115); memória de programa completa e validada; relatório e Trello da 1ª entrega
11/09 – 09/10	Blocos implementados e validados isoladamente; integração no datapath monociclo
09/10 – 13/11	Pipeline completo em simulação, com os três tipos de hazard tratados e o sort validado por completo
13/11 – 11/12	Síntese e testes na FPGA; relatório técnico completo; apresentação final

V. GERENCIAMENTO DO PROJETO NO TRELLO

O board do projeto está organizado em quatro listas: A Fazer → Fazendo → Em Simulação → Concluído. Cada um dos 12 blocos funcionais do processador tem um cartão próprio, com o critério de pronto descrito em checklist: interface definida, Verilog escrito, testbench escrito, simulação aprovada, integração ao top-level e descrição no relatório.

A lista "Em Simulação" foi mantida separada de "Fazendo" para evitar que um bloco com código escrito, mas ainda não validado, seja contado como pronto antes de a simulação confirmar seu funcionamento. Cartões de acompanhamento da entrega atual (tradução do assembly, implementação do montador, redação do relatório e preparação da apresentação) ficam na lista "Fazendo", com prazos e checklist próprios.

Board do Trello: https://trello.com/b/IMyBoCgK/arquitetura-de-sistemas-digitais-mips-sugestão

REFERÊNCIAS

[1] D. A. Patterson e J. L. Hennessy, Computer Organization and Design — The Hardware/Software Interface, 4ª edição revisada. Morgan Kaufmann / Academic Press, 2012.